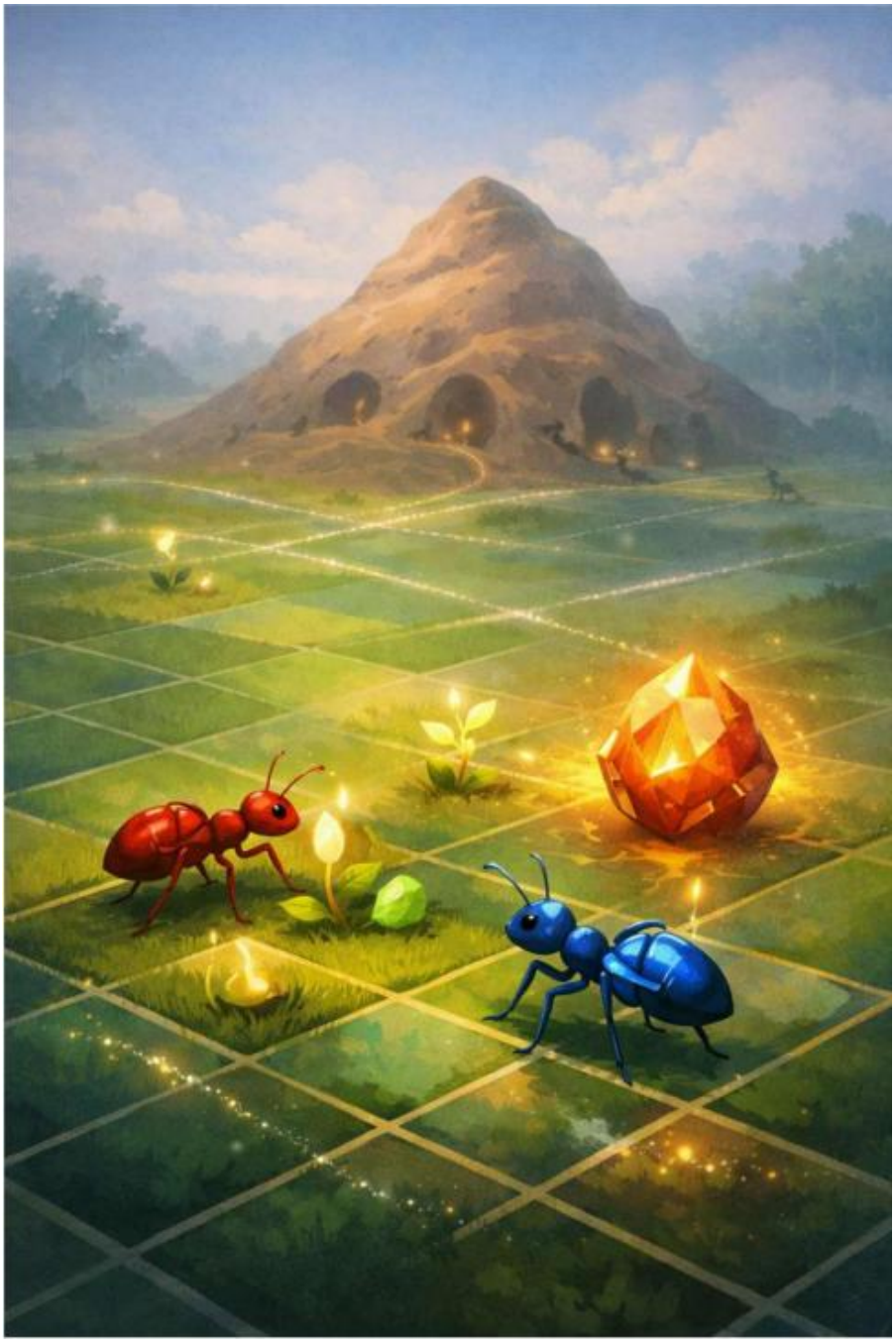
Nom : Azarkan

Prénom : Ilyas

Projet Java III – 2025/2026
HELBFourmiz



Professeur : Riggio Jonathan

Date : 10/08/2026

Table des matières

1. **Introduction**

2. **Fonctionnalités de base**
   - 2.1 Initialisation de la simulation
   - 2.2 Les fourmis
   - 2.3 Les ressources et les graines
   - 2.4 La pierre philosophale
   - 2.5 Le pavage et le système de traces
   - 2.6 Les commandes interactives (Cheat codes)
   - 2.7 Interface graphique

3. **Analyse**
   - 3.1 Architecture MVC
   - 3.2 Organisation du projet
   - 3.3 Hiérarchie des classes
   - 3.4 Cycle de vie d'une ressource

4. **Limitations**
   - 4.1 Déplacements et occupation de la grille
   - 4.2 Sauvegarde
   - 4.3 Évolutions possibles

5. **Conclusion**

6. **Accès LLM**

1. Introduction

Dans le cadre du cours de Java III, il nous a été demandé de développer **HELBFourmiz**, une application simulant le fonctionnement d'une colonie de fourmis évoluant sur une grille. Cette simulation met en scène différents éléments interagissant entre eux, notamment un camp, des fourmis rouges et bleues, des ressources, des graines ainsi que des pierres philosophales. L'objectif principal du projet est de modéliser ces différents éléments en programmation orientée objet tout en respectant les contraintes fonctionnelles et techniques imposées par l'énoncé.

Au cours de la simulation, les fourmis apparaissent progressivement à proximité du camp puis parcourent la grille afin de rechercher des ressources. Une fois une ressource collectée, elles la rapportent automatiquement au camp, ce qui entraîne la création d'une graine. Cette dernière se déplace ensuite vers une zone peu fréquentée avant de germer et de produire une nouvelle ressource. Parallèlement, les déplacements des fourmis laissent des traces visibles sur les cases de la grille, tandis que les pierres philosophales introduisent des événements aléatoires pouvant modifier l'évolution de la colonie.

Le développement de cette application a permis de mettre en pratique les principaux concepts étudiés durant le cours, tels que la programmation orientée objet, l'héritage, le polymorphisme, l'encapsulation, l'utilisation des collections Java, ainsi que la séparation des responsabilités grâce à l'architecture MVC. Une interface graphique a également été développée à l'aide de JavaFX afin de représenter l'évolution de la simulation en temps réel.

Ce rapport présente tout d'abord les principales fonctionnalités implémentées dans l'application, tant d'un point de vue fonctionnel que technique. Il décrit ensuite les principaux choix de conception réalisés ainsi que l'organisation générale du projet. Les limitations de l'application sont ensuite abordées avant de conclure sur les enseignements tirés de la réalisation de ce projet.

2. Fonctionnalités de base

2.1 Initialisation de la simulation

Au lancement de l'application, une nouvelle partie est créée et les différents éléments de la simulation sont initialisés. La classe Game centralise cette logique.

La simulation se déroule sur une grille dont le camp est placé en position (0,0), conformément à l'énoncé. Les ressources initiales ainsi que la première pierre philosophale sont ensuite générées aléatoirement sur des cases libres afin d'éviter toute superposition.

Aucune fourmi n'est présente au démarrage. Elles apparaissent progressivement sur une case libre adjacente au camp jusqu'à atteindre le nombre maximal autorisé, correspondant au nombre de ressources initiales. Si aucune case n'est disponible, leur apparition est temporairement suspendue.

Une fois l'initialisation terminée, la simulation est exécutée grâce à une unique Timeline JavaFX. À chaque rafraîchissement, les fourmis, les graines et les autres éléments mobiles sont mis à jour, ce qui respecte la contrainte imposée d'utiliser une seule boucle d'exécution.

2.2 Les fourmis

Les fourmis sont les principaux acteurs de la simulation. Elles apparaissent progressivement à proximité du camp et recherchent des ressources avant de revenir les déposer au camp.

Deux catégories sont présentes : les fourmis rouges (40 %) et les fourmis bleues (60 %). Chaque fourmi ne peut transporter qu'une seule ressource à la fois.

Les fourmis rouges recherchent systématiquement la ressource la plus proche, tandis que les fourmis bleues se dirigent vers la plus éloignée. Une fois la ressource collectée, elles retournent automatiquement au camp avant de repartir vers un nouvel objectif.

À chaque déplacement, elles laissent une trace sur la case traversée. Les fourmis rouges augmentent la composante rouge de la couleur de la case, tandis que les fourmis bleues modifient uniquement la composante bleue. Ces traces sont ensuite utilisées pour déterminer les futures zones de germination des graines.

Les déplacements peuvent être horizontaux, verticaux ou diagonaux, à condition que la case de destination soit accessible. Des commandes interactives (cheat codes) permettent également de figer les fourmis ou de leur imposer un retour temporaire au camp.

2.3 Les ressources et les graines

Les ressources sont les éléments que les fourmis doivent récolter. Leur nombre initial détermine également le nombre maximal de fourmis pouvant apparaître durant la simulation.

Lorsqu'une ressource est collectée, elle disparaît de la grille et la fourmi retourne automatiquement au camp pour la déposer. Cette collecte incrémente le compteur de ressources rapportées et déclenche la création d'une graine.

Les graines apparaissent à proximité du lieu de collecte puis se déplacent vers une case libre parmi les moins fréquentées par les fourmis. Ce choix est réalisé grâce aux traces laissées sur la grille.

Une fois arrivée à destination, la graine disparaît et une nouvelle ressource apparaît à cet emplacement, assurant ainsi le renouvellement permanent des ressources.

Les graines peuvent également être figées ou générées manuellement grâce aux commandes interactives (cheat codes).

2.4 La pierre philosophale

La pierre philosophale est un élément spécial de la simulation. Elle n'est pas recherchée par les fourmis mais est collectée lorsqu'une fourmi passe sur sa case.

Une seule pierre est présente au lancement du jeu. Son type est choisi aléatoirement parmi les pierres jaune, orange ou rouge, qui possèdent respectivement une probabilité différente de transformer la fourmi en ressource ou de la dédoubler.

Après son utilisation, la pierre disparaît puis réapparaît automatiquement sur une nouvelle case libre avec un type choisi aléatoirement. Une pierre est donc toujours présente sur la carte.

Si une graine traverse une pierre, celle-ci est simplement déplacée sans produire d'effet. Une commande clavier permet également de générer des pierres supplémentaires afin de faciliter les tests.

2.5 Le pavage et le système de traces

La grille est initialisée avec un pavage alternant deux couleurs vertes, conformément aux contraintes de l'énoncé. Les composantes rouge et bleue sont initialement nulles.

Lors du passage d'une fourmi, seule la composante correspondant à son type est augmentée. Les fourmis rouges modifient la composante rouge tandis que les fourmis bleues modifient la composante bleue. Les valeurs sont limitées afin de conserver des couleurs valides.

Au-delà de leur aspect visuel, ces traces permettent de déterminer les zones les moins fréquentées, qui serviront de destination aux graines lors de leur germination.

Des commandes interactives (cheat codes) permettent enfin d'effacer uniquement les traces rouges, uniquement les traces bleues ou l'ensemble des traces de la grille.

2.6 Les commandes interactives (Cheat codes)

Afin de faciliter les tests, plusieurs commandes clavier sont disponibles.

Les touches A, Z, E, R et U permettent de générer respectivement une fourmi rouge, une fourmi bleue, une ressource, une graine ou une pierre philosophale.

Les touches W, X, C et V permettent de figer les fourmis rouges, les fourmis bleues, les graines ou l'ensemble des éléments mobiles.

Les touches J, K et L permettent respectivement de forcer la collecte de toutes les ressources, de faire retourner toutes les fourmis au camp ou de leur faire reprendre leur comportement normal.

Les touches I, O et P réinitialisent les traces laissées sur la grille.

Enfin, la touche M termine la simulation et génère automatiquement le fichier save.jds, contenant les informations demandées par l'énoncé.

2.7 Interface graphique

L'interface graphique a été développée entièrement avec JavaFX. Elle affiche en temps réel la grille ainsi que l'ensemble des éléments de la simulation : camp, fourmis, ressources, graines et pierres philosophales.

Le pavage et les traces sont également représentés visuellement, ce qui permet d'observer les déplacements des fourmis et les zones les plus fréquentées de la carte.

L'affichage est actualisé automatiquement à chaque rafraîchissement de la simulation grâce à la boucle principale. L'utilisateur peut ainsi suivre en continu l'évolution de la colonie et utiliser les différentes commandes clavier sans interrompre l'exécution du programme.

3. Analyse

3.1 Architecture MVC

L'application a été développée selon le modèle d'architecture MVC (Model - View - Controller). Cette organisation permet de séparer les responsabilités du projet et facilite son évolution ainsi que sa maintenance.

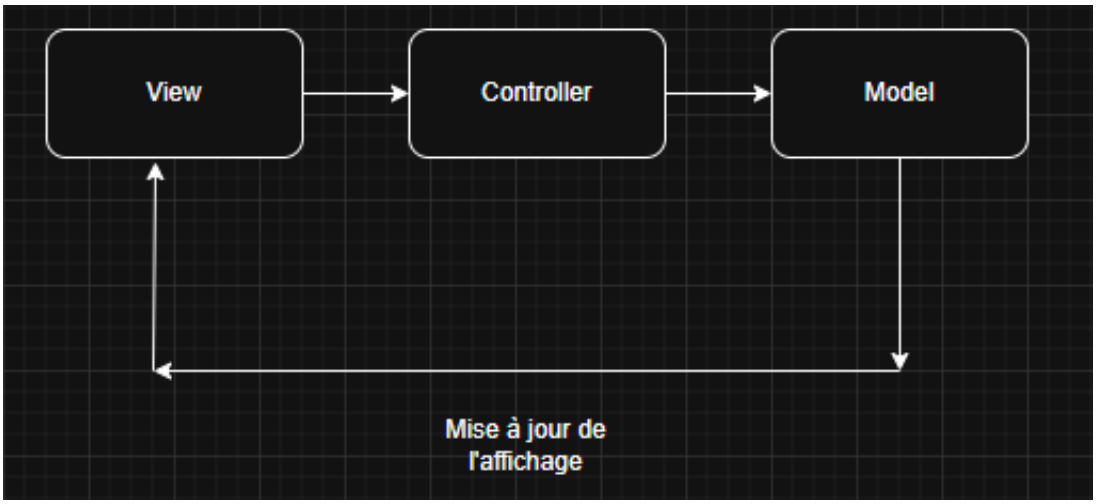
Le Model contient toute la logique métier de la simulation. Il gère les règles du jeu, les déplacements des fourmis, la création des ressources, des graines, des pierres philosophales ainsi que l'ensemble des interactions entre les différents éléments.

Le View est responsable de l'affichage de la simulation. Les classes de ce package dessinent la grille, les entités et les différentes informations visibles par l'utilisateur. Elles ne contiennent aucune logique liée au fonctionnement du jeu.

Le Controller assure la communication entre le modèle et la vue. Il initialise la partie, lance la boucle de simulation et déclenche les mises à jour de l'affichage après chaque rafraîchissement.

Cette séparation permet de modifier plus facilement une partie de l'application sans impacter les autres composants.

Figure 1 : Architecture MVC du projet.



Le contrôleur reçoit les actions de l'utilisateur et les transmet au modèle. Le modèle met ensuite à jour l'état de la simulation. Enfin, la vue est rafraîchie afin d'afficher les nouvelles informations.

3.2 Organisation du projet

Le projet est organisé en plusieurs packages, chacun ayant une responsabilité bien définie.

Le package model regroupe les classes représentant les éléments de la simulation tels que les fourmis, les ressources, les graines, les pierres philosophales, la grille ou encore le camp.

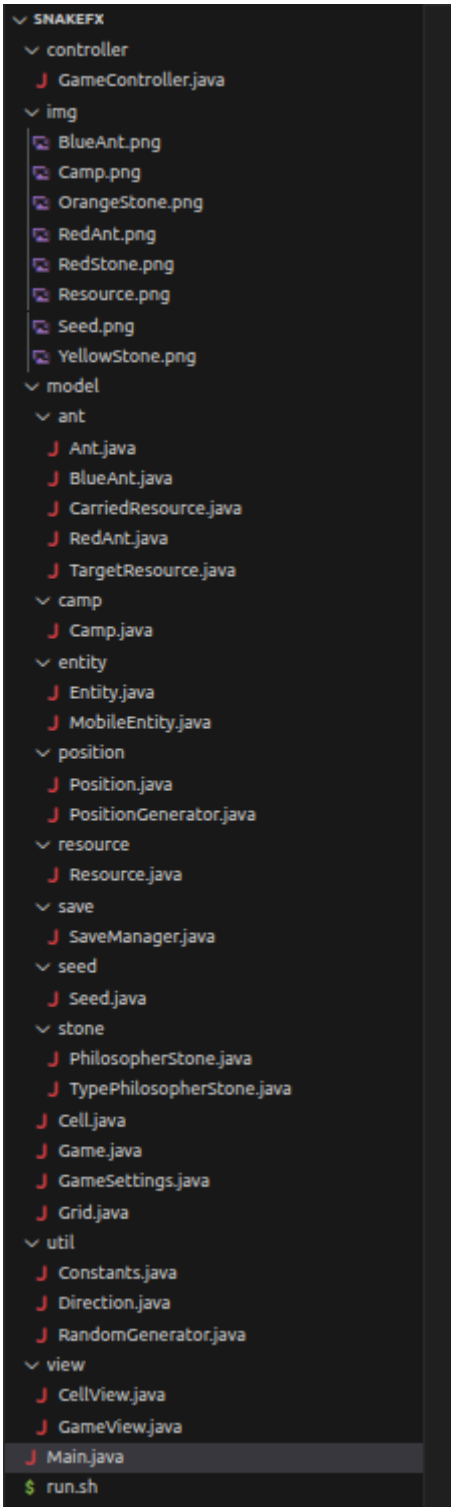
Le package view contient les classes responsables de l'affichage graphique avec JavaFX.

Le package controller assure le pilotage général de la simulation et la communication entre le modèle et l'interface graphique.

Le package util rassemble les classes utilitaires, notamment les constantes utilisées dans l'ensemble de l'application afin d'éviter les valeurs magiques.

Cette organisation améliore la lisibilité du code et facilite son évolution.

Figure 2 : Organisation des packages du projet.



Le projet est organisé en plusieurs packages afin de séparer les responsabilités. Le package model contient toute la logique de la simulation, view gère l'affichage avec JavaFX, controller fait le lien entre le modèle et la vue, tandis que util regroupe les classes utilitaires utilisées dans l'ensemble de l'application

3.3 Hiérarchie des classes

L'application utilise l'héritage afin de partager les comportements communs entre plusieurs classes.

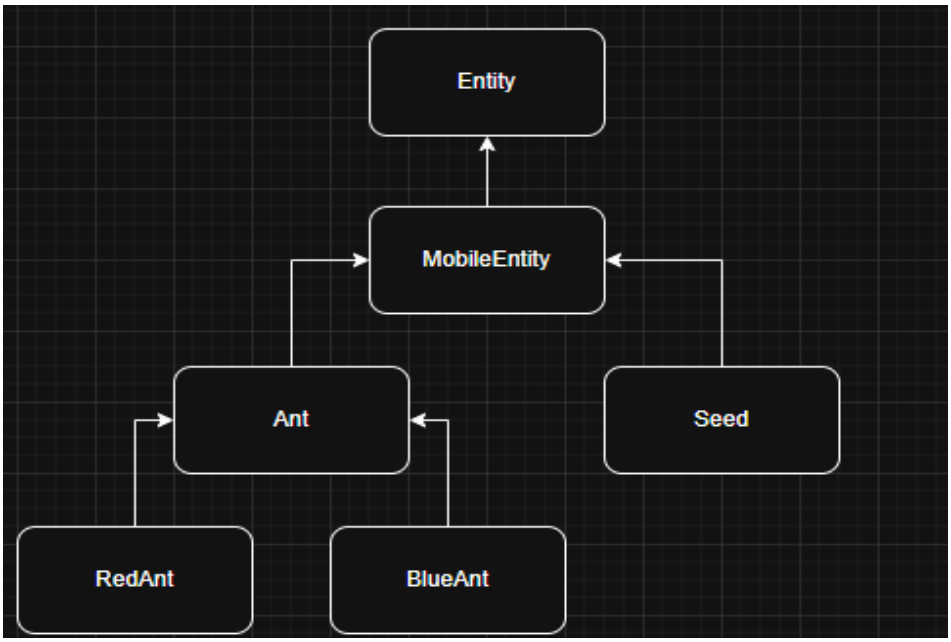
Les différentes entités de la simulation héritent d'une classe de base représentant les caractéristiques communes, comme la position sur la grille. Les entités mobiles héritent ensuite d'une classe intermédiaire ajoutant les fonctionnalités de déplacement.

Les classes RedAnt et BlueAnt héritent toutes les deux de la classe abstraite Ant, qui regroupe le comportement commun des fourmis. Chaque sous-classe implémente ensuite sa propre stratégie de recherche des ressources.

Les différentes pierres philosophales utilisent également l'héritage afin de partager un comportement commun tout en définissant leurs probabilités propres.

Cette hiérarchie permet de limiter la duplication de code et facilite l'ajout de nouveaux types d'entités.

Figure 3 : Diagramme simplifié des principales classes.



La conception de l'application repose sur l'héritage afin de factoriser le code commun. Les entités mobiles héritent de MobileEntity, qui hérite lui-même de Entity. Les classes RedAnt et BlueAnt héritent de Ant et implémentent chacune un comportement de recherche de ressources différent. La classe Seed hérite également de MobileEntity car elle possède les mêmes mécanismes de déplacement.

3.4 Cycle de vie d'une ressource

Le cycle de vie d'une ressource constitue l'un des principaux mécanismes de la simulation.

Une ressource est d'abord créée lors de l'initialisation ou après la germination d'une graine. Une fourmi la recherche ensuite selon son comportement, la collecte puis retourne au camp afin de la déposer.

Le dépôt de cette ressource provoque la création d'une graine. Celle-ci se déplace vers une zone peu fréquentée avant de disparaître et de faire apparaître une nouvelle ressource.

Ce fonctionnement permet d'assurer un renouvellement permanent des ressources tout au long de la simulation.

4. Limitations

4.1 Déplacements et occupation de la grille

Les déplacements des entités reposent sur un système relativement simple. À chaque mise à jour, une fourmi ou une graine se rapproche de son objectif en avançant d'une case horizontale, verticale ou diagonale. Avant chaque déplacement, une vérification est effectuée afin de s'assurer que la case de destination est accessible.

Ce fonctionnement est suffisant pour répondre aux exigences du projet, mais il présente certaines limites. Lorsqu'une case est temporairement occupée par une autre entité

mobile, la fourmi doit attendre qu'elle se libère avant de poursuivre son déplacement. Aucun algorithme de recherche de chemin plus avancé n'est utilisé pour contourner automatiquement cet obstacle.

Dans une version plus évoluée, il serait possible d'utiliser un algorithme de recherche de chemin (pathfinding). Ce type d'algorithme permet à une entité de déterminer automatiquement un autre trajet lorsqu'un obstacle bloque son déplacement, au lieu d'attendre que la case se libère. Les déplacements seraient ainsi plus fluides et plus efficaces, en particulier lorsque le nombre de fourmis présentes sur la grille devient important.

4.2 Sauvegarde

L'application réalise une sauvegarde automatique lorsque la simulation est arrêtée à l'aide de la commande prévue. Le fichier save.jds contient le nombre de fourmis rouges, le nombre de fourmis bleues ainsi que le nombre total de ressources rapportées au camp, conformément aux exigences de l'énoncé.

Cette sauvegarde reste volontairement limitée. Elle ne permet pas de reprendre une partie exactement dans l'état où elle a été interrompue. Les positions des entités, les traces présentes sur la grille, les ressources restantes ou encore les graines en déplacement ne sont pas enregistrées.

Une amélioration possible consisterait à sauvegarder l'ensemble de l'état de la simulation afin de pouvoir reprendre une partie ultérieurement sans perte d'informations. Cela nécessiterait cependant une gestion plus complète de la sérialisation des différents objets du projet.

4.3 Évolutions possibles

Plusieurs améliorations pourraient être apportées afin d'enrichir la simulation sans remettre en cause son architecture.

Il serait tout d'abord possible d'ajouter de nouveaux types de fourmis possédant des comportements différents, comme des fourmis capables de coopérer ou de modifier leur stratégie en fonction de la situation. De nouveaux types de pierres philosophales ou de ressources pourraient également être imaginés afin de rendre la simulation plus variée.

L'interface graphique pourrait également être enrichie. Par exemple, des boutons permettant d'exécuter les différentes commandes interactives (cheat codes) pourraient compléter les raccourcis clavier afin de rendre l'application plus intuitive pour l'utilisateur.

Enfin, il serait intéressant d'ajouter des paramètres configurables avant le lancement de la simulation, comme la taille de la grille, le nombre initial de ressources, la vitesse de rafraîchissement ou encore la proportion de fourmis rouges et bleues. L'application gagnerait ainsi en flexibilité tout en conservant la même architecture générale.

5. Conclusion

Le développement de HELBFourmiz m'a permis de mettre en pratique les principaux concepts étudiés durant le cours de Java III. La réalisation de ce projet m'a amené à concevoir une application complète en programmation orientée objet, en utilisant notamment l'héritage, le polymorphisme, l'encapsulation ainsi que l'architecture MVC afin de séparer la logique métier de l'interface graphique.

L'ensemble des fonctionnalités principales demandées dans l'énoncé a été implémenté. La simulation permet notamment la gestion des fourmis, des ressources, des graines, des pierres philosophales, du système de traces, des commandes interactives (cheat codes) ainsi que de la sauvegarde des informations demandées en fin de simulation.

Au-delà des aspects techniques, ce projet m'a également permis de mieux comprendre l'importance d'une bonne organisation du code. Une architecture claire, des

responsabilités bien réparties entre les classes et l'utilisation de constantes facilitent la maintenance, les corrections et les évolutions futures de l'application.

Enfin, même si plusieurs améliorations pourraient encore être apportées, notamment concernant les déplacements des entités ou les possibilités de configuration de la simulation, ce projet répond aux objectifs fixés et constitue une bonne mise en pratique des notions abordées durant le cours.

6. Accès LLM

Email : azarkan.ilyas.java.2526.2s@gmail.com

Mot de passe : Ladiqekuj.2006

https://chatgpt.com/share/6a73a384-ce28-83eb-b32c-29ce0593801e
https://chatgpt.com/share/6a73a392-4afc-83ed-b644-b2be8069b169
https://chatgpt.com/share/6a73a39f-ceb4-83eb-a976-15d966f60092